

CS3611

计算机网络大作业

2026 Spring

题目五：基于 UDP 的应用层可靠传输与 AI 驱动拥塞控制协议实现

第 XX 组（成员姓名与学号待最终核对）

小组成员

序号	姓名 / GitHub 账号	学号
1	蒋昊翔 / Lawrencilia	524030910083
2	dumeili114514	待补
3	gggggs1	待补
4	liujin / hero-always	待补

2026 年 6 月 7 日

1 项目概述

1.1 实验目标

本项目模拟 QUIC 将可靠传输和拥塞控制上移到应用层的思想, 基于 Python 原生 UDP Socket 实现带有确认、重传、滑动窗口和拥塞窗口控制的可靠传输协议。在此基础上, 项目构建虚拟瓶颈链路, 对比传统 AIMD 与 Q-Learning、DQN 两类智能拥塞控制策略在吞吐、时延和重传方面的表现。

1.2 完成情况总览

表 1: 功能完成情况

类别	功能点	状态
基础功能	UDP Payload 中封装 Sequence Number、Timestamp 与 1024 字节 Payload	✓
	接收端累计 ACK、发送端未确认队列、RTO 定时重传与 SRTT 采样	✓
	虚拟瓶颈链路: 固定服务速率、有限队列、排队延迟与队列溢出丢包	✓
	AIMD 拥塞控制基线: ACK 加性增、超时或快速重传时乘性减	✓
	Q-Learning 控制器: 6 状态、3 动作、在线 Q-Table 更新与可视化	✓
扩展功能	带宽中途减半, 观察控制器对动态网络突变的响应	✓
	连续 3 个重复 ACK 触发快速重传, 接收端缓存乱序分组	✓
	DQN 深度强化学习控制器, 使用 PyTorch 模型输出 CWND 乘数动作	✓
未选择项	学生自定义开放性问题: 本组未选择该可选扩展, 本文不计入完成范围	未选

1.3 需求覆盖与提交物清单

除自定义开放性问题外, 题目五基础功能和已选扩展均已完成。代码、实验数据和文档提交物对应关系如下。

表 2: 题目要求与仓库文件对应关系

要求项	实现与证据	提交物
应用层可靠传输	protocol.py 定义 SEQ+Timestamp+Payload 与 ACK; sender.py 维护 unacked、ACK worker、timer worker、RTO 重传、RTT/SRTT; receiver.py 返回累计 ACK	protocol.py, sender.py, receiver.py
虚拟瓶颈链路	VirtualFunnellink 在 sendto() 外层实现有限 FIFO 队列、固定服务间隔、队列满丢弃和链路统计	virtual_link.py
AIMD 与 Q-Learning	sender.py 中 CongestionController 支持 fixed/aimd/qlearning; Q-Learning 使用 6 状态、3 动作、 ϵ -greedy 和 Bellman 更新	q_table.json, train_q_learning.py, artifacts/training/
数据记录与可视化	Sender 输出 artifacts/metrics/metrics.csv 与 artifacts/metrics/history.csv; plot_metrics.py 生成 CWND、吞吐和 RTT 对比图; 报告包含 Q-Learning 多轮趋势图	artifacts/metrics/, plot_metrics.py, report/figures/*.png
动态突变、快重传、DQN	--link-bandwidth-drop-after-packets 模拟中途带宽减半; 3 个重复 ACK 触发 FAST 重传; DQN 用连续状态和 PyTorch 网络输出 CWND 乘数	dqn_model.pt, train_dqn.py, artifacts/demo_results/
演示与文档	一键演示依次运行 AIMD、Q-Learning、DQN 和带宽突变场景, 保存日志、CSV 和图表; 本报告与 poster 使用同一批结果	Run_Demo.command, demo_runner.py, report/output/

1.4 模块与分工

分工依据本地 Git 提交记录、分支和 GitHub PR 记录整理。PR #1 (finish project5) 合并了可靠传输、虚拟链路、AIMD/Q-Learning、CSV 记录与可视化主线; PR #2 (Add DQN-based congestion control and training script) 合并了 DQN 扩展。自动化 review / conflict-fix 机器人只作为辅助工具, 不计入小组成员贡献。

表 3: GitHub commit 级别分工明细

成员 / 账号	主要证据	具体工作	产出
Lawrencilia / Haoxiang Jiang	6a688ac, 1ac1838, efe154b, 06282cd, 124c984, PR #1 管理	初始化仓库与 README; 建立可靠传输底层, 补齐 SEQ/Timestamp/Payload 封装、发送端未确认队列、ACK 接收线程、RTO timer、RTT/SRTT 采样; 实现/整合快速重传和乱序处理说明; 重构实验脚本, 修复 Q-Table、metrics、history 输出目录问题; 整合报告、poster 和一键演示。	protocol.py; sender.py; receiver.py; README; demo/report/poster
dumeili114514	92d880d, dc95a4e	设计早期 Q-Learning 智能拥塞控制核心: 6 个 RTT 趋势/丢包离散状态、保持/加一/减半 3 个动作、奖励函数、 ϵ -greedy 探索与 Bellman 更新; 编写多轮 Q-Learning 训练脚本; 补入 AIMD 基线, 在 ACK 时加性增、拥塞信号时乘性减, 并配合 benchmark/绘图验证。	congestion_control.py; train_q_learning.py; Q-Learning/AIMD 逻辑
ggggs1	228a9ae, e790d8d, PR #1 分支	实现虚拟网络拓扑模拟: VirtualFunnellink 有限 FIFO 队列、固定服务间隔、队列溢出丢包、链路统计和日志; 将虚拟链路参数接入 Sender; 完成题目五收尾文档, 添加 plot_metrics.py 以及 AIMD/Q-Learning 对比图, 支撑报告中的实验数据与可视化。	virtual_link.py; plot_metrics.py; required comparison 图; 实验提交文档
liujin / hero-always	b7cf010, PR #2	完成 DQN 扩展: 在 sender.py 增加 dqn 模式、连续状态 $[RTT, loss\%, CWND]$ 、状态归一化、动作乘数 (0.50, 0.75, 1.00, 1.25, 1.50)、经验回放、SmoothL1 loss、target network、模型加载/保存和 DQN 日志; 编写 train_dqn.py 多轮训练脚本, 提交模型权重与 DQN 对比实验图。	train_dqn.py; dqn_model.pt; DQN comparison 图; artifacts/training/

2 系统设计

2.1 整体架构

系统由 Sender、Receiver、应用层协议封装、虚拟瓶颈链路、拥塞控制器和数据记录流水线组成。Sender 产生固定大小分组并按当前 CWND 控制在途包数量; VirtualFunnellink 在真实 sendto() 之前模拟固定带宽和有限缓存; Receiver 维护期望序号并返回累计 ACK; Sender 根据 ACK 采样 RTT、推进窗口并触发智能控制器。

UDP Reliable Transport with AI Congestion Control

Application-layer reliability, virtual bottleneck link, AIMD/Q-Learning/DQN adaptive CWND

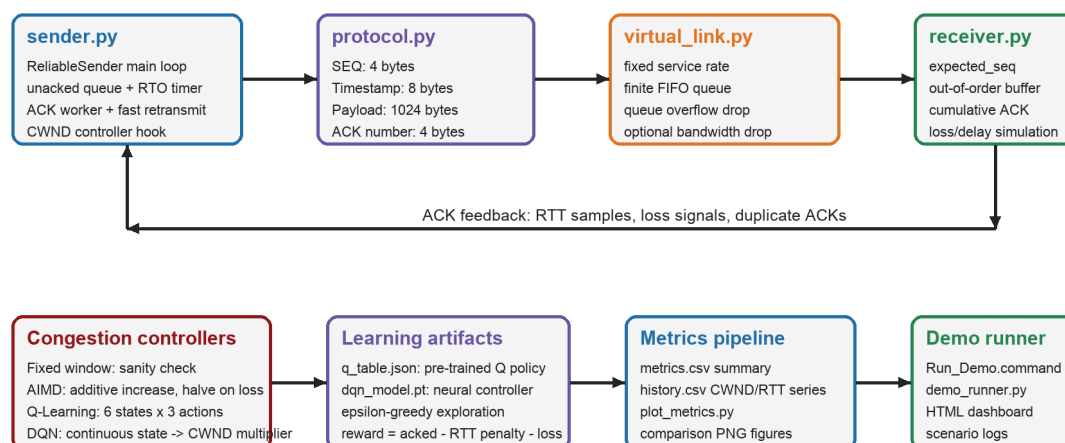


图 1: 系统架构与数据反馈路径

2.2 应用层 UDP 协议

表 4: 自定义数据包与 ACK 格式

字段	长度	说明
Sequence Number	4 Bytes	无符号整数，标识分组序号
Timestamp	8 Bytes	发送端写入 Unix 时间戳，用于 RTT 采样
Payload	1024 Bytes	固定长度数据负载
ACK Number	4 Bytes	接收端返回的最大连续确认序号，-1 表示尚无连续分组

协议由 `protocol.py` 实现，核心格式为 `"!Id"` 与 `"!i"`。其中 `!` 表示网络字节序，`I` 为 4 字节无符号整数，`d` 为 8 字节浮点时间戳。每次新发或重传都会重新封包，以保证 `Timestamp` 反映本次上链路的时间。

2.3 可靠传输与虚拟链路

发送端用 `unacked` 字典保存未确认分组的 `Payload`、最近发送时刻、写入报文的时间戳和发送次数。主线程只在 `len(unacked) < cwnd` 时发送新分组；ACK 线程收到累计 ACK 后删除所有 `seq <= ack_number` 的记录，并根据被确认分组的 `Timestamp` 计算 `RTT` 与 `SRTT`；Timer 线程周期扫描 `unacked`，超过 `RTO` 即重传。

虚拟瓶颈链路包装 Sender 的发送动作。应用层数据先进入容量有限的 `FIFO` 队列，再由后台线程按固定服务间隔发往真实 `UDP Socket`。若队列已满，虚拟链路直接丢弃新分组并记录丢包日志；若启用带宽突变参数，链路会在转发指定包数后调大服务间隔，模拟带宽突然下降。

3 核心实现

3.1 AIMD 基线

AIMD 模式复现 TCP Reno 的拥塞避免思想。初始化 $CWND = 1$ ；每收到新 ACK，执行

$$CWND \leftarrow \min(CWND_{\max}, CWND + \frac{newly_acked}{CWND})$$

当 RTO 或快速重传表示出现拥塞信号时，执行

$$CWND \leftarrow \max(1, CWND/2)$$

因此 AIMD 曲线应呈现“逐步上升、拥塞后骤降、再重新上升”的锯齿形态。

3.2 Q-Learning 智能拥塞控制

Q-Learning 控制器每个控制周期统计 ACK 数、平均 RTT、重传次数和 RTO timeout 次数，使用 6 个离散状态：

$$State = \{RTT_up, RTT_down, RTT_stable\} \times \{loss, no_loss\}$$

动作空间为 0 保持、1 表示 $CWND + 1$ 、2 表示 $CWND/2$ 。调参后的奖励函数使用 timeout-heavy 惩罚：

$$R = \alpha G - \beta N_{timeout} - \gamma N_{retr} - \delta RTT_{ms}$$

其中 G 为控制周期内成功确认的分组数，可作为 goodput 代理； $N_{timeout}$ 为 RTO 超时次数； N_{retr} 为总重传次数； RTT_{ms} 为控制周期平均 RTT。默认权重为 $\alpha = 1.0$ 、 $\beta = 10.0$ 、 $\gamma = 2.0$ 、 $\delta = 0.015$ 。由于 timeout 会同时计入 $N_{timeout}$ 和 N_{retr} ，单次 RTO 默认扣分明显高于 fast retransmit，更符合 timeout 对性能破坏更大的常识。每轮结束后按 Bellman 公式更新 Q 值：

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

训练后的表格策略保存到 `q_table.json`，运行时可通过 `--epsilon 0.0` 使用预训练策略进行稳定评估。

3.3 DQN 扩展

DQN 模式将离散 Q-Table 扩展为神经网络控制器。输入为 RTT、丢包率和当前 CWND 组成的连续状态，输出 5 个动作的 Q 值；动作对应 0.50 到 1.50 的 CWND 乘数。模型由三层全连接网络实现，并通过经验回放和 target network 训练，权重保存为 `dqn_model.pt`。该扩展用于探索连续网络状态下更细粒度的拥塞响应。

4 测试与结果

4.1 测试环境与方法

实验使用本地环回地址 127.0.0.1，每轮传输 300 个 UDP 数据分组，Receiver 端加入少量随机丢包、延迟和抖动，Sender 启用虚拟瓶颈链路。Q-Learning 和 DQN 训练脚本默认进行 50 轮训练，并建议正式复现实验时使用 50–100 轮观察趋势。所有模式默认输出到 `artifacts/metrics/` 或 `artifacts/training/`，再由 `plot_metrics.py` 在 `artifacts/plots/` 生成 CWND、吞吐量和 RTT 对比图。一键演示脚本 `demo_runner.py` 会依次运行 AIMD、Q-Learning、DQN 以及带宽突变场景，并将完整结果保存到 `artifacts/demo_results/`。

表 5: 2026-06-04 一键演示实验结果

模式	确认	耗时 (s)	吞吐 (Mbps)	平均 RTT(ms)	SRTT(ms)	重传	快速重传
AIMD	300/300	4.113	0.597	89.783	114.360	25	18
Q-Learning	300/300	6.375	0.385	19.183	44.648	14	0
DQN	300/300	6.880	0.357	18.102	12.714	9	0
Q-Learning 带宽减半	300/300	6.674	0.368	66.715	59.180	38	16

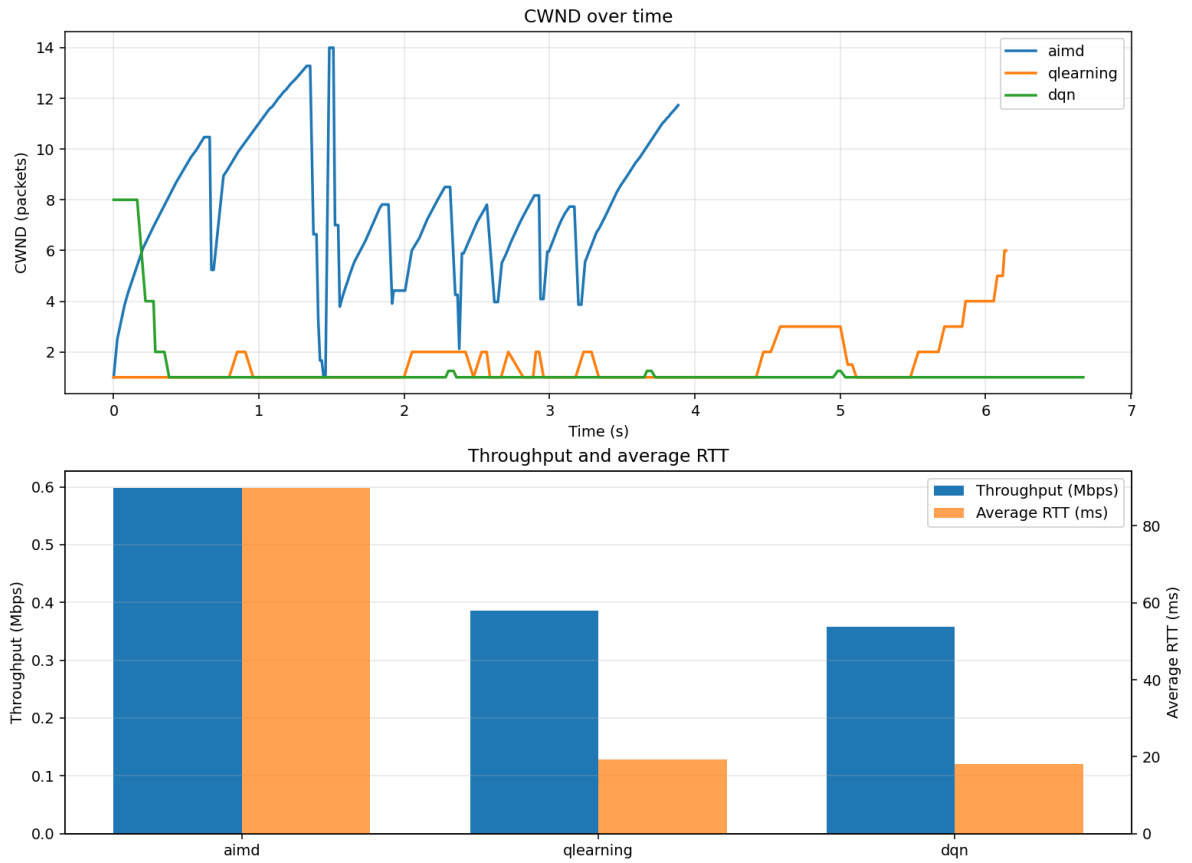


图 2: AIMD、Q-Learning 与 DQN 的 CWND、吞吐量和平均 RTT 对比

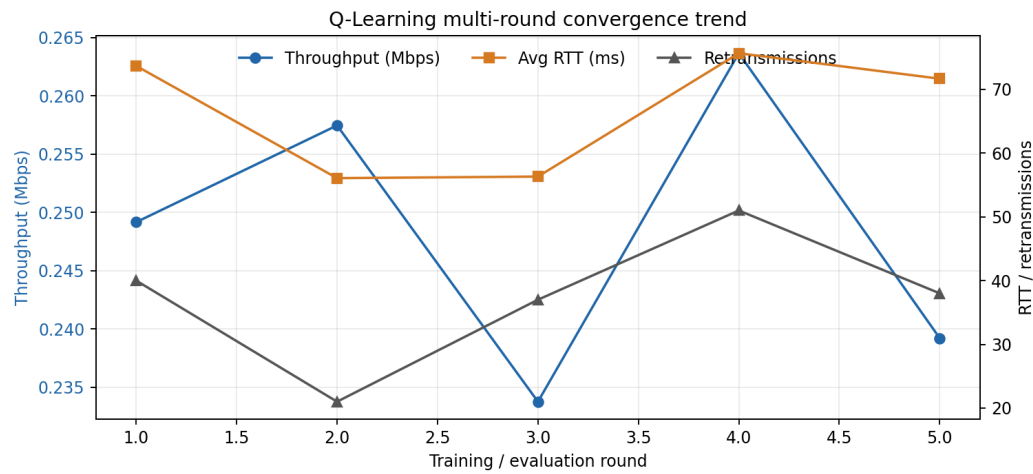


图 3: Q-Learning 多轮训练/评估趋势：吞吐、平均 RTT 与重传次数

4.2 结果分析

三种模式均完成 300/300 分组可靠交付，说明应用层 ACK、重传、乱序处理和累计确认机制有效。AIMD 的吞吐量最高，为 0.597 Mbps，但平均 RTT 达 89.783 ms，且发生 25 次重传，其中 18 次由快速重传触发；这与其主动探测带宽、不断推高 CWND 的设计一致。

Q-Learning 与 DQN 更保守，牺牲部分吞吐换取低时延与更稳定的队列状态。Q-Learning 平均 RTT 为 19.183 ms，DQN 平均 RTT 为 18.102 ms，均显著低于 AIMD。DQN 的重传次数进一步降至 9 次，说明连续状态输入和 CWND 乘数动作在本次实验条件下能保持较低排队延迟。图 3 使用训练指标 CSV 中的多轮 Q-Learning 记录绘制，展示了训练/评估过程中吞吐、RTT 和重传指标的变化；正式提交时推荐运行 50 或 100 轮，以得到更有说服力的收敛趋势。

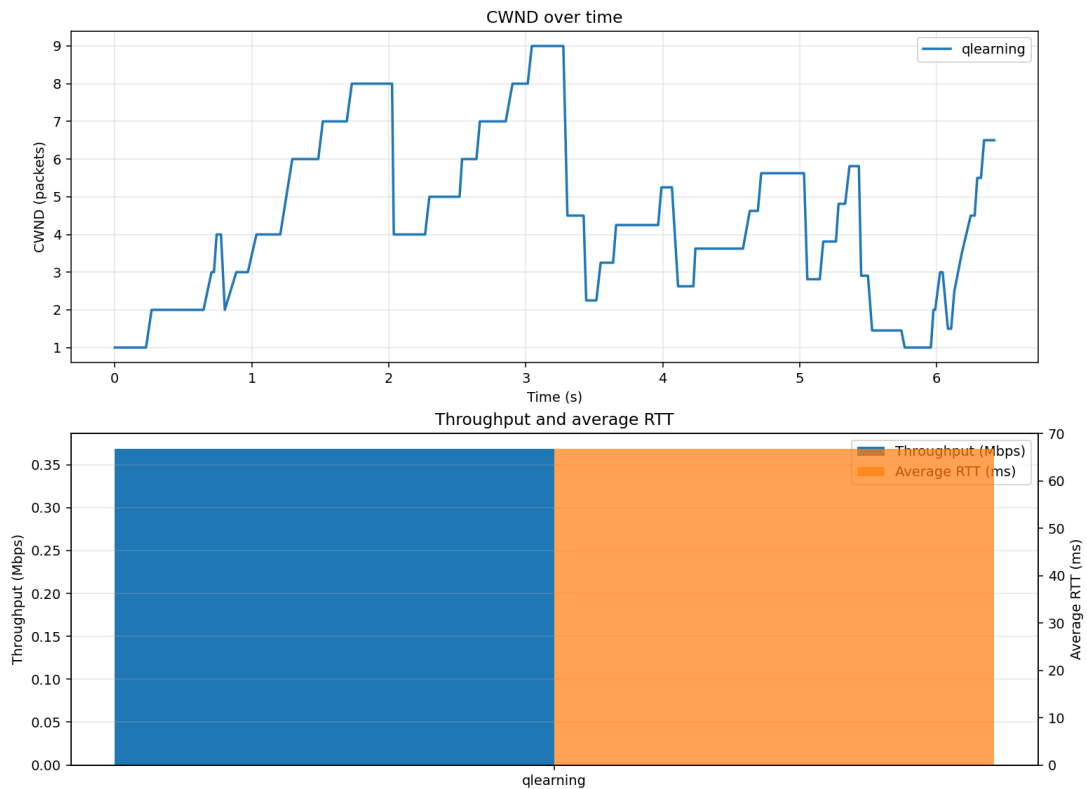


图 4: Q-Learning 在带宽中途减半场景下的恢复过程

带宽减半场景中，Q-Learning 仍完成所有分组交付，但平均 RTT 上升到 66.715 ms，重传增加到 38 次。其 CWND 曲线在突变后出现下降和再探索，表明虚拟链路突变扩展能够真实触发拥塞信号，也能用于现场演示不同控制策略的恢复过程。

5 操作手册

5.1 环境准备

核心可靠传输、AIMD 与 Q-Learning 只依赖 Python 标准库；训练进度条需要 `tqdm`，绘图需要 `matplotlib`，DQN 需要 `torch`。

Listing 1: 依赖安装

```
python3 -m pip install tqdm matplotlib torch
```

5.2 运行方式

最简方式是在 macOS 双击 `Run_Demo.command`，或在项目目录运行：

Listing 2: 一键演示

```
1 python3 demo_runner.py
```

手动运行时，先启动 Receiver，再在另一个终端运行 Sender：

Listing 3: 手动运行

```
1 python3 receiver.py --host 127.0.0.1 --port 9001 --initial-seq 0
2 python3 sender.py --target-host 127.0.0.1 --target-port 9001 \
3   --packets 300 --cc-mode qlearning --window-size 1 \
4   --max-cwnd 80 --qtable-file q_table.json \
5   --metrics-file artifacts/metrics/metrics.csv \
6   --history-file artifacts/metrics/history.csv
```

生成对比图：

Listing 4: 绘图

```
1 python3 plot_metrics.py --metrics-file artifacts/metrics/metrics.csv \
2   --history-file artifacts/metrics/history.csv \
3   --output artifacts/plots/comparison.png
```

6 总结

本项目完成了题目五要求的 UDP 应用层可靠传输、虚拟瓶颈链路、AIMD 基线、Q-Learning 智能拥塞控制和实验可视化，并实现了动态带宽突变、快速重传与乱序处理、DQN 深度强化学习三项扩展。实验结果表明，AIMD 更擅长快速抢占吞吐，但容易推高 RTT 并触发更多重传；Q-Learning 和 DQN 能以更保守的 CWND 控制显著降低时延，在应用层协议上体现出 AI 驱动拥塞控制的稳定性优势。